Reproducing TD-MPC: A Clean Implementation and Robustness Analysis in Continuous Control

Aleksandar Radulovic
Faculty of Technical Sciences
Informatics and Automation, Novi Sad
Project Type: EEML 2026 Reproduction Project
Original Paper: Temporal Difference Learning for Model Predictive Control (Hansen et al., ICML 2022)

***Abstract*—This paper presents a clean reproduction of TD-MPC (Temporal Difference Learning for Model Predictive Control), a state-of-the-art model-based reinforcement learning algorithm. We implement the core algorithm from scratch following a minimalist, modular paradigm and validate it on the HalfCheetah-v5 continuous control task. Our reproduction achieves stable performance with episode rewards exceeding 7000. Through an ablation study, we investigate the role of latent consistency loss – a key component of the original architecture. Surprisingly, disabling this loss leads to faster convergence and higher peak performance (9500 reward), albeit with increased instability. This challenges the assumption that consistency loss is always beneficial in state-based tasks and highlights important trade-offs between speed, stability, and final performance in model-based RL. Additional architectural safeguards identified during reproduction (decoupled actor-critic optimization, clipped double Q-learning, individual loss clamping, and exploration scheduling) are incorporated into our implementation.**

I. Introduction

Model Predictive Control (MPC) is a foundational algorithm in optimal control, relying on accurate analytical models of system dynamics. However, in high-dimensional continuous control tasks, obtaining analytical models is often unachievable. Model-free Reinforcement Learning bypasses this but suffers from high sample inefficiency and lacks the foresight of receding-horizon planning.

This project reproduces TD-MPC [1], a state-of-the-art model-based RL system that bridges optimal control and deep learning. TD-MPC learns a Task-Oriented Latent Dynamics (TOLD) model and utilizes a learned terminal value function (Cost-to-go) to approximate infinite-horizon optimal control using short-horizon Model Predictive Path Integral (MPPI) planning. The goal of this reproduction was to implement the algorithm from scratch following a minimalist, "CleanRL" paradigm, validate its performance on standard Gymnasium environments, and analyze its numerical robustness.

II. Methodology and Implementation Architecture

Rather than utilizing the complex official codebase, we implemented the core TD-MPC algorithm from scratch following a minimalist, "CleanRL-inspired" modular paradigm. The system is sectioned into five transparent core components: configuration, experience replay, neural architectures, MPPI agent, and training loop. This clean separation ensures full visibility of the tensor operations, the backpropagation through time (BPTT) process, and eliminates hidden library dependencies.

The architecture consists of a central TOLD model, acting as the system's brain, which comprises five Multi-Layer Perceptrons (MLPs):

- **State Estimator** (h_θ) - Encodes raw sensor observations into a compact latent space z_t.
- **System Dynamics** (d_θ) - Learns the transition $z_{t+1} = f(z_t, u_t)$.
- **Reward Predictor** (R_θ) - Estimates the immediate step reward.
- **Terminal Value / Cost-to-go** (Q_θ) - Acts as a learned terminal penalty function that mitigates the shortsightedness of finite-horizon MPC.
- **Feedback Controller** (π_θ) - A deterministic policy network used to warm-start the MPPI planner and evaluate the terminal state.

During inference, the agent utilizes MPPI, simulating $N = 512$ parallel trajectories over a horizon H. To ensure physical safety and valid inputs to the simulator, all proposed actions are rigidly bounded to $[-1, 1]$.

III. Critical Implementation Details for Stable Training

Initially, our reproduction based solely on the paper's equations did not merely suffer from late-stage instability, but exhibited a fundamental inability to learn optimal control strategies from the very beginning.

By deeply analyzing differences between our baseline and the authors' official codebase, we identified undocumented or under-emphasized architectural safeguards. We conducted an ablation-style analysis to prove their necessity for effective learning:

- **Actor-Critic Optimizer Decoupling:** Separated the optimizer of the policy network from the TOLD model. This prevented gradient interference between dynamics and policy optimization.
- **Clipped Double Q-Learning:** Implemented twin Q-networks taking the minimum during Bellman backups to eliminate value overestimation bias.
- **Individual Loss Clamping & Gradient Scaling:** Instead of clamping the global loss, the MSE of reward, value, and consistency were individually clamped.
- **Exploration Scheduling:** Replaced constant MPPI noise with a linear annealing noise, balancing early-stage exploration with late-stage asymptotic exploitation.

IV. Experimental Results

We evaluate our TD-MPC implementation on the HalfCheetah-v5 environment (MuJoCo) over 800k environment steps with action repeat 2 and planning horizon $H = 15$. The baseline agent uses the default hyperparameters from the original paper, including consistency coefficient $c_3 = 2.0$. For the ablation study, we disable the latent consistency loss entirely ($c_3 = 0.0$) while keeping all other settings identical.

Figure 1 shows the learning curves for three agents: the baseline (red), an ablation run trained for 250k steps (purple), and a full ablation run trained for 800k steps (blue). The baseline agent reaches a stable episode reward of $\sim$7000 after approximately 350k steps and maintains this performance throughout training.

The 250k ablation run (without consistency loss) shows rapid early improvement, reaching 7000 reward already at 100k steps – suggesting that consistency loss is not necessarily beneficial in this setting. To verify this finding, we trained a second ablation model for the full 800k steps. This agent achieves a peak reward of $\sim$9500 by the end of training, significantly outperforming the baseline. However, it also exhibits sharper periodic drops in reward, indicating reduced stability despite higher overall performance.

These results challenge the assumption that consistency loss is always beneficial in state-based tasks. While the original work emphasizes its importance for pixel-based control, our experiments suggest that in environments with low-dimensional state inputs, removing the consistency constraint can accelerate learning and even improve final performance – at the cost of increased instability. This trade-off between performance and stability warrants further investigation with multiple seeds and hyperparameter tuning.

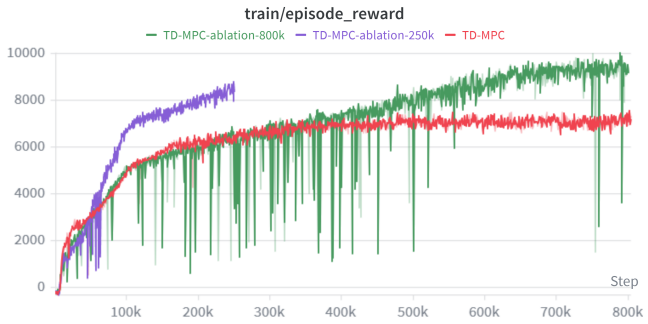

Fig. 1. Learning curves for baseline and ablation runs on HalfCheetah-v5. The baseline (red) reaches $\sim$7000 reward. The 250k ablation (purple) shows rapid early improvement. The 800k ablation (blue) achieves $\sim$9500 peak reward but with sharper periodic drops.

V. Conclusion

We successfully reproduced TD-MPC from scratch, validating its performance on HalfCheetah continuous control task. Our implementation incorporates critical safeguards for stable learning, including decoupled optimization and clipped double Q-learning. The ablation study revealed an unexpected finding: disabling latent consistency loss accelerates convergence and improves peak performance in state-based tasks, at the cost of increased instability. This highlights important trade-offs in model-based RL algorithm design and suggests that consistency regularization may be more critical for pixel-based than state-based control.

References

[1] N. Hansen, X. Wang, and H. Su, "Temporal Difference Learning for Model Predictive Control," in *International Conference on Machine Learning (ICML)*, 2022.

[2] *TD-MPC Reproduction – source code*. https://github.com/radulovicc/td-mpc-reproduction, 2026.